

SoC May 2026

WEEK 2: Introduction to Reinforcement Learning

Mayuresh Thengzode
25B1816

Algorithm Overview & MDPs (Lectures 1-2)

1. Q-Learning Overview

Q-learning is a model-free, off-policy temporal difference (TD) reinforcement learning algorithm used to find the optimal action-selection policy for a Markov Decision Process (MDP). It operates by learning an action-value function, denoted as $Q(s, a)$, which estimates the total expected future reward an agent can obtain by taking a specific action a in a given state s . The agent explores the environment and iteratively updates its Q-values using the Bellman Optimality Equation:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

where α is the learning rate, γ is the discount factor, and R is the immediate reward.

Because it is off-policy, Q-learning updates its estimates based on the absolute best possible future action rather than the action actually chosen by the current exploration policy (such as ϵ -greedy). In environments with small, discrete state and action spaces, these values are stored and maintained in a look-up table known as a Q-table. As the agent continuously interacts with the environment, the Q-values asymptotically converge toward the true optimal values, allowing the agent to select the highest-rewarding actions.

2. Monte Carlo (MC) Methods

Unlike Temporal Difference methods, Monte Carlo methods learn directly from **episodes of raw experience** without modeling the environment's dynamics. MC approaches compute value estimates by averaging the total actual returns received after visiting a state. Because updates are only performed at the very end of an episode (when a terminal state is reached), MC estimates are unbiased but suffer from high variance, making them less suitable for exceptionally long or infinite-horizon tasks.

3. Proximal Policy Optimization (PPO)

PPO is a popular **on-policy actor-critic** algorithm designed to make policy gradient methods more stable and data-efficient. It introduces a specialized clipped surrogate objective function that penalizes policy updates that move too far from the old policy:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

This clipping mechanism prevents destructively large policy steps, ensuring reliable, monotonic performance improvements across both discrete and continuous action spaces.

4. Deep Deterministic Policy Gradient (DDPG)

DDPG is an **off-policy, model-free actor-critic** algorithm specifically tailored for continuous action spaces. It concurrently learns a deterministic policy (the actor) and an action-value function (the critic). To bridge the gap between continuous control and deep learning, DDPG utilizes an experience replay buffer to break sample correlation and employs slowly updating **target networks** ($\theta^- \leftarrow \tau\theta + (1 - \tau)\theta^-$) to stabilize the Q-learning targets.

5. Soft Actor-Critic (SAC)

SAC is an **off-policy, maximum entropy actor-critic** framework designed for continuous control. What sets SAC apart is its objective function, which encourages the agent to maximize not only its expected cumulative reward but also its **entropy** (randomness/exploration):

$$J(\pi) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t) \sim \rho_\pi} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))]$$

By actively seeking out policy variety, SAC prevents premature convergence to local optima, achieves exceptional sample efficiency, and exhibits high robustness to environment changes.

6. Short Derivation of Bellman Expectation Equations

The total return G_t from time-step t is defined recursively as $G_t = R_{t+1} + \gamma G_{t+1}$. Using the law of total expectation, we can derive the recursive relationship for both $V(s)$ and $Q(s, a)$.

Derivation for State-Value Function $V^\pi(s)$

$$\begin{aligned} V^\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma V^\pi(S_{t+1}) \mid S_t = s] \end{aligned}$$

Explicit expansion:

$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} P(s' \mid s, a) [R(s, a, s') + \gamma V^\pi(s')]$$

Derivation for Action-Value Function $Q^\pi(s, a)$

$$\begin{aligned} Q^\pi(s, a) &= \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma Q^\pi(S_{t+1}, A_{t+1}) \mid S_t = s, A_t = a] \end{aligned}$$

Explicit expansion:

$$Q^\pi(s, a) = \sum_{s' \in S} P(s' \mid s, a) \left[R(s, a, s') + \gamma \sum_{a' \in A} \pi(a' \mid s') Q^\pi(s', a') \right]$$

Glossary of Core Reinforcement Learning Terms

Model-Free

Algorithms that learn directly through interaction without requiring a transition probability model.

Off-Policy

Learning about a target policy while following a different behavior policy.

On-Policy

Evaluating and improving the same policy used to make decisions.

Temporal Difference (TD)

Learning after every step by bootstrapping estimates.

Actor-Critic

Architecture splitting the agent into a policy optimizer (Actor) and value estimator (Critic).

Experience Replay Buffer

Storing transitions in memory to sample random batches for training.

Maximum Entropy RL

Optimizing for both reward and policy randomness to ensure thorough exploration.